

On the Combination of PID control and Reinforcement Learning: A Case Study with Water Tank System

Yuting Wu, Lantao Xing, Fanghong Guo, and Xiaokang Liu

Abstract: Reinforcement learning (RL) has attracted great interest from researchers in recent years. RL performs as human or better in many fields such as games and robot control. Although this technology is booming in computer science, it has not been practically applied in industrial process control. Up to now, proportional–integral–derivative (PID) control is still the most dominating and popular control method in industrial control. In this paper, we propose a combination of deep reinforcement learning (DRL) and PID control for better process control performance. The idea is generated by the following observations: for PID controller, its transient performance is not usually well enough to meet a strict requirement or in complex signal tracking tasks; For RL technology, a perfectly designed reward function is required for training. However, in practice, the reward function needs to be tested through trial and error, which will lead to a waste of computational power and time. By combining these two strategies, PID controller can help to improve the steady-state performance of RL control by its integral term, while the trained RL agent is able to improve the transient performance of PID controller. Several case studies with the water tank system are presented to demonstrate the effectiveness of the combined PID + RL control strategy.

Key Words: PID, Reinforcement learning, Water tank system

1 Introduction

The two-level water tank control system, with a simple model structure and a clear control objective, is one of the most commonly used and tested models in process control area. We can get a deeper understanding of the control performance of different control strategies through this classical system. The control object is to maintain the water level of the second tank at the set level. The control input is the open ratio of the water pump which decides the quantity of input water sent to the first water tank. Both the first and second tanks are gravity-drained water tanks and they are linked via a valve.

The easiest way to manage the water tank system is proportional–integral–derivative (PID) control, as it is simple and easy to implement. However, compared with advanced control strategies (e.g. Model Predictive Control [1]), sometimes PID control precision is not good enough. It has some limitations in dealing with complicated non-linear systems and complex signal tracking tasks. The tedious tuning problem for the PID controller is almost inevitable to meet the desired control requirement. The tuning process is to find three specific parameters, and then linearly combine these three proportional–integral–derivative components. Engineers have developed many methods, such as Ziegler-Nichols [2] (Z-N), first-order-plus-dead-time [3] (FOPDT) approximation, to find a set of relatively reasonable parameters. However, it is unknown that whether a linear combination of these three PID components is sufficient enough.

Reinforcement learning (RL) can also be used as a tool to design controllers by solving the Bellman function [4]

This work was supported by Nanyang Technological University under the Presidential Postdoctoral Fellowship Program (*Corresponding author: Lantao Xing*, E-mail: lantao.xing@ntu.edu.sg).

Y. Wu and L. Xing are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore (Email: yuting.wu@ntu.edu.sg).

F. Guo is with the Department of Automation, Zhejiang University of Technology, Hangzhou 310032, China.

X. Liu is with the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, 430074, China.

through “error and trial”. It differs from supervised and unsupervised machine learning technologies in that there is no prior knowledge or training data required. It learns from the concept of “reward” designed by designers and learns to take optimal actions from historical state-action-reward experiences. By interacting with the black-box environment, it is able to find the optimal solution considering the long-term reward. A typical RL agent consists of a policy function, a state value function, and a state-action value function which help to achieve the doing, evaluating, and learning goals.

In recent year, the deep machine learning (DML) and artificial intelligence (AI) tools have achieved great success in various fields (e.g. image process [5] and data analysis [6]), which have drastically changed people’s way of life compared with the decades ago, ranging from face-scanning payment to self-driving cars. With the help of deep neural network (DNN), deep reinforcement learning (DRL) shows its priority beyond the human level [7]. Alpha Go Zero from Deep Mind Lab, defeated Ke Jie, one of the greatest Go players around the world, in 3 out of 3 games in 2017 with only 21 days of training. Besides games, DRL is also applied in more complicated tasks in [8]. Several DRL algorithms have been developed and widely used these years such as DQN in Atari game [9], A3C [10] and PPO [11].

Despite these achievements in the computer science area, DRL is still commonly considered not reliable enough in control area because of the lack of theoretical proof and the use of randomly initialized neural networks. Moreover, DRL technology also has the following disadvantages: 1). It requires massive computational power and time to interact with the environment in order to generate enough state-action-reward training data sets; 2). The equipment or components of the environment may be damaged if a bad action is taken during training; 3). The random seed selection in the code may cause uncertain and unstable training results; 4). The design of the reward is critical for the training process [12] as a sparse reward in most tasks will lead to inefficient learning in the initial phase of training, which is time-wasting; 5). It is difficult to find a perfect reward design in the trade of training efficiency and accuracy. In a safety-

sensitive industrial process control environment, there is still a long way to go for DRL.

Based on the aforementioned discussions, in this paper we propose to combine the deep deterministic policy gradient (DDPG) [8] algorithm and PID controller together to get better control performances. DDPG is an RL algorithm with continuous action space and deterministic policy. The DDPG agent aims to provide a supplementary control signal to the PI controller. Specifically, the PI controller is first tuned by FOPDT approximation method; Then, the DDPG agent is trained with the PI controller as guidance. It is shown that, after the combination, PI controller can help to improve the steady-state performance of RL control by its integral term, while the trained RL agent is able to improve the transient performance of the PI controller. Three case studies with the two-level water tank system illustrate the advantages of the combination of PID and RL.

The rest of the paper is organized as follows. The control environment and control objective is explained in Section 2. The DDPG algorithm is illustrated in Section 3. The reward and state design for the DDPG agent is shown in Section 3.1. In Section 4, three case studies are provided to validate the proposed control strategy. The conclusion will be made in Section 5.

2 Problem Formulation

The two-level water tank system is a common system in industrial control. The model of the l -th tank is

$$\dot{h}_l(t) = \frac{1}{A} [F_{in_l}(t) - F_{out_l}(t)], \quad l = \{1, 2\},$$

where h_l is the height of the water level, A is the cross-sectional area of the tank, F_{in_l} and F_{out_l} represent the quantity of input and output water flow, respectively. Both tanks are gravity drained as their output water flow speeds are related to their own height of water level.

The two-level water tank system is composed of two single water tanks with self-balancing ability connected in series as shown in Fig. 1. The control objective is to maintain the water level l_c in the second tank at the desired level l_d .

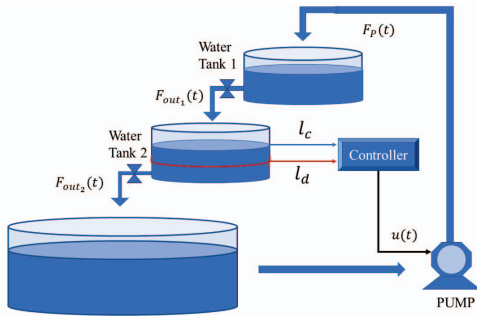


Fig. 1: Two-level water tank system.

For the first tank, the input pump flow and gravity drained output flow are defined as:

$$\begin{aligned} F_p(t) &= b \cdot u(t) \\ F_{out_1}(t) &= a \cdot \sqrt{h_1(t)}, \end{aligned}$$

where a and b are the parameters related to flow rate, and $u(t)$ the opening ratio of the water pump.

Then we can generate the model for Tank 1:

$$\dot{h}_1(t) = \frac{1}{A} [F_p(t) - F_{out_1}(t)].$$

For the second tank, its input water flow is the output of water tank 1 and its gravity drained output flow have a same structure as $F_{out_1}(t)$ as follows:

$$\begin{aligned} F_{in_2}(t) &= F_{out_1}(t) \\ F_{out_2}(t) &= a \cdot \sqrt{h_2(t)}. \end{aligned}$$

The model of the water tank 2 is:

$$\dot{h}_2(t) = \frac{1}{A} [F_{out_1}(t) - F_{out_2}(t)].$$

The two-level water tank systems generally exhibit second-order characteristics. Nevertheless, in RL training, we regard it as a black-box model in this paper.

Since there is a time lag through adjusting the pump ratio of Tank 1 to control the water level in Tank 2, it is difficult to maintain the precise liquid level manually. In order to achieve the process control goals, the PI controller is commonly used. We first fit the control system with a First-Order-Plus-Dead-Time (FOPDT) model by a unit step response. Through FOPDT optimizing, we can generate a continuous PI controller as formula (1):

$$u(t) = K_p e(t) + K_i \int_0^t e(t) dt, \quad (1)$$

where K_p and K_i are Proportional gain and Integral gain, respectively. The output of the PI controller is saturated as the open ratio of the water pump, and a Band-Limited White Noise is added to imitate mechanical disturbance in the water pump.

3 Deep Deterministic Policy Gradient (DDPG) Algorithm

RL algorithm does not require accurate model parameters as it treats the control environment as a black box. RL algorithm interacts with the environment by giving an action and receiving the associated response on the evaluation of the action. Among several RL algorithms, DDPG algorithm [8] is specified for the case of continuous action space. In the two-level water tank problem, we need to control the pump opening ratio within the range of $[0, 1]$, which is a continuous action space. Hence we adopt the DDPG algorithm to train the RL controller. Let us give a briefing about DDPG. "Deep" implies using DNNs, $Q(s, a|\theta^Q)$ and $\mu(s|\theta^\mu)$ represent critic and actor function approximation, respectively; "Deterministic" implies that with a given state s , it takes a deterministic action a instead of a probability distribution, namely in the form of

$$a = \mu(s),$$

where μ is the actor function. For the action space exploration, a random process $\mathcal{N}_t$ is added on policy μ as follows:

$$a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t,$$

where the subscript t denotes the current step and θ^μ is the parameter of actor-network. The critic-network $Q(s, a|\theta^Q)$ is used to evaluate how good the policy is, and the actor-network $\mu(s|\theta^\mu)$ generates action a_t once receiving the current state s_t . In the beginning, the critic-network and actor-network will be randomly initialized with weights θ^Q and θ^μ , and then they will be updated to cover the target network Q' and μ' accordingly.

At each step, the actor takes action a_t based on pre-action state s_t under current policy μ_t . The reward r_t is calculated based on the post-action state s_{t+1} . The (s_t, a_t, r_t, s_{t+1}) transients will be stored into the replay buffer R for training two networks. A random mini-batch of N transitions (s_i, a_i, r_i, s_{i+1}) will be sampled from replay buffer R to train the critic agent through minimizing the following loss:

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2,$$

where y_i is the sum of the reward r_i and γ -discounted expected long-term reward under current policy starting from the next time step:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^{Q'}).$$

A greater discount factor γ attaches more importance to the long-term reward.

In addition, the actor policy will be updated by the sampled policy gradient to find the optimal policy that can maximize the long-term reward as follows:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu)|_{s_i}.$$

Then, the network parameter of actor and critic will be updated by:

$$\begin{aligned} \theta^\mu &= \theta^\mu + \alpha \nabla_{\theta^\mu} J \\ \theta^Q &= \theta^Q - \beta \nabla_{\theta^Q} L, \end{aligned}$$

where α and β are the learning rates of the network. At the end of each episode, the target networks will be soft-updated as follows:

$$\begin{aligned} \theta^{\mu'} &\leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \\ \theta^{Q'} &\leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}, \end{aligned}$$

where τ is the learning target smooth factor.

Repeat the above process until the maximum step T or meet the predefined stop condition in each episode. The training will be ended if it meets our desired control performance or reaches the maximum of episode M . The algorithm is concluded in algorithm 1.

Existing methods use DDPG as only control input in the water tank system. In this paper, we use DDPG as a supplemental control signal of PID control. The design of state and reward functions will be shown in the next subsections.

3.1 DDPG Based Control Agent Design in water Tank System

In this subsection, we will show the design of the DDPG agent in the two-level water tank system. First, we replace

Algorithm 1 DDPG algorithm in water tank system

Initialization

```

Initialize the replay buffer  $R$ .
Initialize the critic-network  $Q(s, a|\theta^Q)$  and the actor-network  $\mu(s|\theta^\mu)$  with random weights.
Initialize the target network  $Q'$  and  $\mu'$  with copied network parameters.
for episode = 1,  $M$  do
    Initialize the desired water level  $l_d$ .
    Initialize the water level  $l_c$  in Tank 2.
    Generate the initial state  $s_1$ .
    for step = 1,  $T$  do
        Run the simulation.
        Take action  $a_t$  based on current policy  $\mu_t$  and exploration noise  $\mathcal{N}_t$ .
        Observe the post-action state  $s_{t+1}$ .
        Calculate the reward  $r_t$ .
        Store the  $(s_t, a_t, r_t, s_{t+1})$  transient into replay buffer  $R$ .
        Randomly Sample  $N$  transitions  $(s_i, a_i, r_i, s_{i+1})$  from  $R$ .
        Update the critic by minimizing the loss.
        Update the actor policy by maximizing the expected long-term reward.
        Soft-update the target network.
        if water level  $l_{c_t} < 0$  or  $l_{c_t} > 4$  then
            break
        end if
    end for
end for

```

the PI controller with an RL agent to apply the DDPG algorithm to control the water tank level. The whole system is shown in Fig. 2.

Then we need to design the state and the reward function for DDPG agent. The observations from two-level water tank system are the integrated error between desired water level and current water level in Tank 2, error and current height of the water tank as shown in formula 2:

$$\text{state} = \left[\int (l_d - l_c) dt, l_d - l_c, l_c \right], \quad (2)$$

where l_d and l_c are the desired water level and current water level at each step respectively.

The reward function design is intuitive as the negative absolute value of the error signal at each step. If the water level in Tank 2 is not in the range of $[0, 4]$, the current episode will be immediately stopped and given a large penalty of -500 . The following equation 3 is used to calculate the reward value:

$$\text{reward} = \begin{cases} -|l_d - l_c| & 0 \leq l_c \leq 4 \\ -500 & \text{else.} \end{cases} \quad (3)$$

The DDPG agent is trained to maximize the accumulated rewards in T steps.

4 Simulation Analysis

In this section, the comparison between the PI controller and DDPG agent will first be given. In the training process, the water level in Tank 1 will be initialized to 0 meters, the desired and the current water level in Tank 2 will be initialized to a random value between $[0.5, 1.5]$ and $[0, 4]$ respectively at the beginning of each episode. There is no

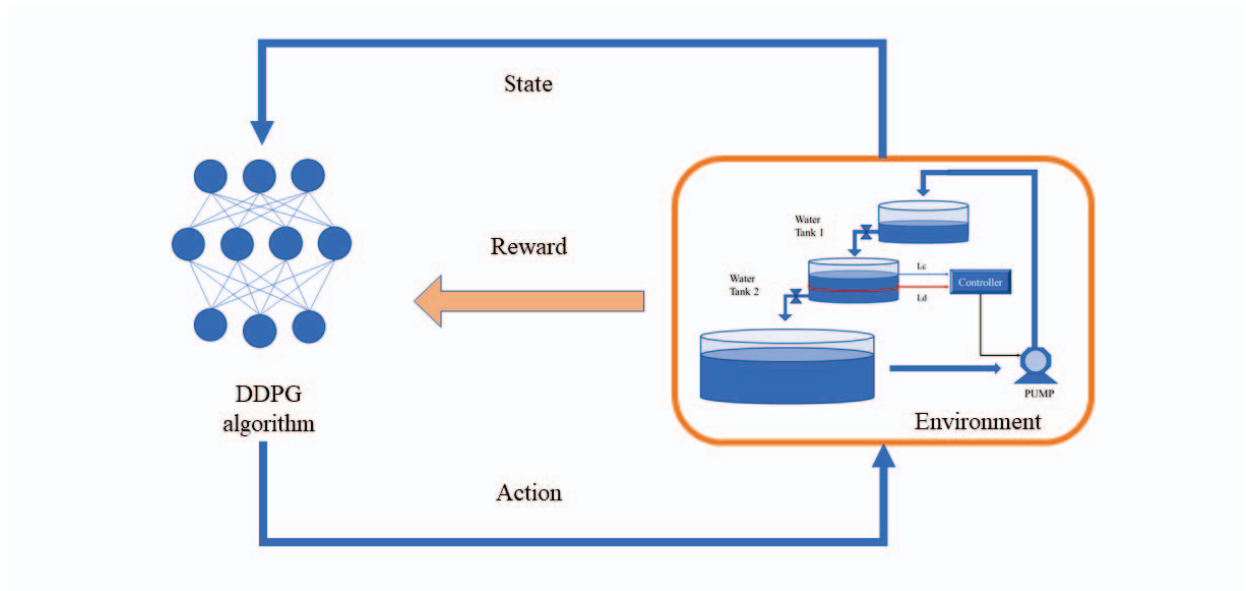


Fig. 2: Two-level water tank system with DDPG agent.

disturbance added during the training process. The training parameters of the DDPG agent, PI controller and the water tank system are given in Table 1. Then we will validate our proposed PI + DDPG control strategy in several cases.

Table 1: Model Parameters

Replay Buffer R	$1e5$
Mini-batch Size N	64
Episode M	300
Max Step T	200
Agent Sample Time T_s	1 sec
Agent Noise $\mathcal{N}$ Variance	0.3
Agent Noise $\mathcal{N}$ Variance Decay Rate	$1e-05$
Actor Learning Rate α	$1e-04$
Critic Learning Rate β	$1e-03$
Discount Factor γ	1
Target Smooth Factor τ	$1e-03$
Proportional Gain K_p	2
Integral Gain K_i	0.03
Water tank cross-sectional area A	1
Valve parameter a	0.04
Valve parameter b	0.08

4.1 DDPG Agent and PI Controller Comparison

The initialization of the model is required for each episode. The reset liquid level in the second tank ranges from $(0, 3]$ and the desired water level ranges from $[0.5, 1.5]$ meters. The white noise is not added during training.

The training process lasts 300 episodes and the average accumulated rewards finally converged to -60. At the initial 50 episodes of training, the average accumulated rewards are lower than -200.

Fig. 3 shows the comparison results of the DDPG agent and the PI controller. Compared with the sole PI control, the transient response speed is improved by DDPG agent. For the PI controller, it takes more than 400 seconds to stabilize

with an overshoot of 43% at the first swing. However, the stable time for the DDPG agent is only about 250 seconds and the largest overshoot amplitude is only 12.6%. This illustrates that the control performance of the DDPG agent is more smooth and efficient.

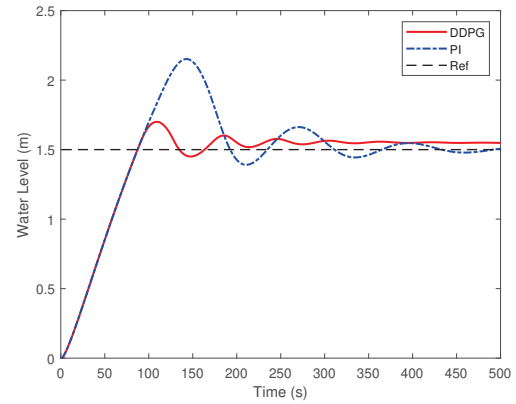


Fig. 3: Comparison result between DDPG and PI controller.

However, in the steady-state, from Fig. 3, it is seen that a derivation error always exists for the DDPG control result. Residual is the weakness of the DDPG agent which could be eliminated by the integral term in the PI controller or a carefully designed reward function. In fact, the reward design is very tricky and time-consuming because its performance cannot be known in advance unless hours of training are conducted. In many cases, a seemingly perfect reward could significantly slow down the training process and the agent can learn nothing in the first hundreds of episodes[12]. To improve the control performance and the training efficiency, we propose the combined control strategy using the DDPG agent as a supplementary control signal in addition to the PI

controller.

4.2 PI control + DDPG agent

In this section, we will validate our proposed control strategy through three cases. The control signals from the PI controller and DDPG agent will be summed and then be saturated between $[0, 1]$. The schematic diagram of PI + DDPG control is shown in Fig. 4. The control signal from the DDPG agent is restricted in the range of $[-1, 1]$ to avoid it from dominating the combined signal. The simulations are realized on Matlab 2020b and the training process lasts about 7000 seconds for 300 episodes on GTX 1070 GPU and i7-8750H CPU.

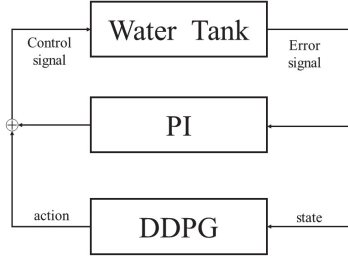


Fig. 4: The schematic diagram of PI + DDPG control

In the following, 3 different case studies will be discussed.

4.2.1 Case A: Basic Case Study

To investigate the control performance of the PI + DDPG control strategy, the same initialization is made as those in Section 4.1.

After 300 episodes of training, the DDPG agent has converged and the average accumulated rewards in each episode have reached -30. With the guidance from the PI controller, the training process of the DDPG agent is accelerated. The average accumulated reward of PI + DDPG controller in the training process starts from -90. However, it takes more than 70 episodes of training in Section 4.1 to reach the same level of average accumulated rewards.

The control result is shown in Fig. 5. The overshoot amplitude of the PI + DDPG controller is 21.8%, which is half of the value of the sole PI controller. Moreover, the weakness of the sole DDPG controller is fixed as the water level converges exactly to the desired value.

4.2.2 Case B: Noise Case Study

To investigate the noise tolerance of our proposed strategy, a noise case is studied in this subsection. In Case B, the Band-Limited White Noise block is added with Noise Power parameter set as 0.1 and Sample time set as 1. The noise is added before control signal saturation and it is used to simulate the random mechanical error between the controller and the pump. It is worth mentioning that the noise case is not enabled during the DDPG agent training process.

In this study, three control strategies are compared in Fig. 6 with the same initialization. In the sole PI control case, the noise influence is not very obvious before the first swing. Instead, it has more impact on the steady-state per-

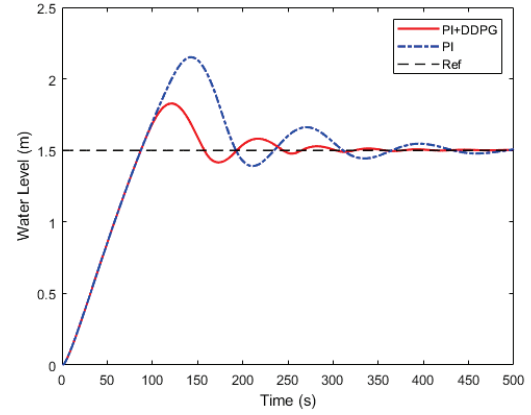


Fig. 5: Comparison between PI + DDPG and PI in Case A.

formance. By contrast, in the case of the DDPG agent only, the adverse effect of noise is conspicuous and it affects more on the transient performance. The derivation at steady state is still as mentioned before.

In comparison, the PI + DDPG control strategy demonstrates a strong anti-disturbance ability. The control performances with and without the added noise are similar. Again, even in the presence of noise, the combined strategy still shows better performance compared with the sole PI and DDPG agent.

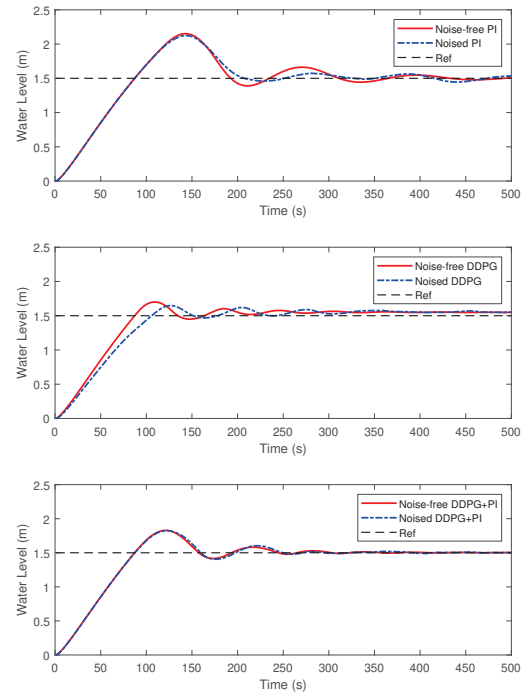


Fig. 6: Noise Case Study.

4.2.3 Case C: Changing Desired Water Level

This part provides another experiment to analyze these three control strategies under the case where the desired water level may switch.

As shown in Fig. 7, the desired water level is initialized at 1.5 meters and then switched to 2.5 meters at the 500th second, and finally switched back after 1000 seconds. In the training process, the desired water level is ranging from 0.5 to 1.5 meters and the switching case in this experiment is not included.

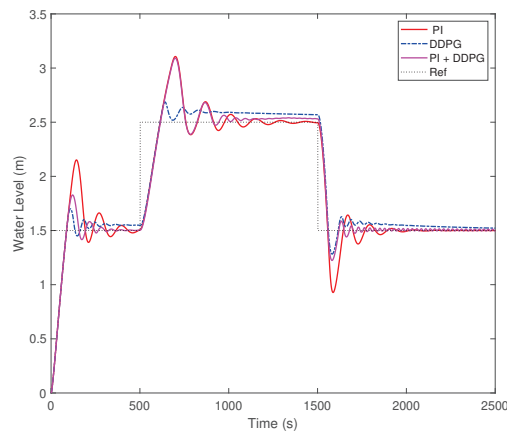


Fig. 7: Case with changing desired water level.

In comparison to the sole DDPG agent and the sole PI controller, although this switching situation has not been learned yet during the RL training process, the DDPG agent is still able to greatly reduce the overshoot when the reference water level switches. However, the derivation error in the steady-state still exists and becomes larger in the middle term of the simulation.

The PI + DDPG controller has a different control pattern from the DDPG or PI controller. At the first stage, the PI + DDPG controller trades off between overshoot and accuracy as illustrated in Case A. And at the second stage where the desired water level is switched to 2.5 meters, the combined controller decides to first follow the PI control signal at the untrained part and then slowly participates in control. At the final stage, where this situation has been learned before, the controller shows an improved control performance again.

5 Conclusion

This paper has proposed a novel PID + RL control strategy. The performance and effectiveness of the control strategy has been tested through a two-level water tank system. It has shown that the proposed combined control strategy achieves a significant improvement in the transient performance compared with the PI controller. In the meantime, it can also significantly reduce the training time and exclude the steady-state error of sole RL control. This combination of PID and RL control strategies demonstrates great potential in process control area.

References

- [1] C. E. García, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—a survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.
- [2] J. G. Ziegler, N. B. Nichols *et al.*, "Optimum settings for automatic controllers," *trans. ASME*, vol. 64, no. 11, 1942.
- [3] Q. Bi, W.-J. Cai, E.-L. Lee, Q.-G. Wang, C.-C. Hang, and Y. Zhang, "Robust identification of first-order plus dead-time model from step response," *Control Engineering Practice*, vol. 7, no. 1, pp. 71–77, 1999.
- [4] R. Bellman, *Dynamic Programming*. Dover Publications, 1957.
- [5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [6] J. Qiu, Q. Wu, G. Ding, Y. Xu, and S. Feng, "A survey of machine learning for big data processing," *EURASIP Journal on Advances in Signal Processing*, vol. 2016, no. 1, pp. 1–16, 2016.
- [7] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing atari with deep reinforcement learning," *CoRR*, vol. abs/1312.5602, 2013.
- [8] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016.
- [9] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nature*, vol. 550, pp. 354–, Oct. 2017.
- [10] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," *CoRR*, vol. abs/1602.01783, 2016.
- [11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017.
- [12] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," 2017, cite arxiv:1709.06560Comment: Accepted to the Thirtieth-Second AAAI Conference On Artificial Intelligence (AAAI), 2018.